

RESEARCH ARTICLE | OCTOBER 10 2025

Predicting drag coefficients of vehicle geometries: A PointNet++ point cloud surrogate model approach

Xiaoyu Qi ; Ming Yang; Guang Tong 

AIP Advances 15, 105015 (2025)

<https://doi.org/10.1063/5.0300798>

Articles You May Be Interested In

TripNet: Learning large-scale high-fidelity three-dimensional car aerodynamics with triplane networks

Physics of Fluids (June 2026)

DrivAer transformer: A high-precision and fast prediction method for vehicle aerodynamic drag coefficient based on the DrivAerNet++ dataset

Physics of Fluids (July 2025)

Point cloud data enhancement and automatic topology structure extraction technology for complex distribution network scenes based on deep learning

AIP Advances (March 2026)



Special Topics Open for Submissions

[Learn More](#)

Predicting drag coefficients of vehicle geometries: A PointNet++ point cloud surrogate model approach

Cite as: AIP Advances 15, 105015 (2025); doi: 10.1063/5.0300798

Submitted: 4 September 2025 • Accepted: 29 September 2025 •

Published Online: 10 October 2025



Xiaoyu Qi,^{a)}  Ming Yang,^{b)} and Guang Tong^{c)}

AFFILIATIONS

Shanghai Dianji University, Shanghai, China

^{a)}24600002150302@st.sdju.edu.cn

^{b)}yangm@sdju.edu.cn

^{c)}Author to whom correspondence should be addressed: tongg@sdju.edu.cn. Current address: School of Mechanical Engineering, Shanghai Dianji University, Shanghai, China.

ABSTRACT

In automotive aerodynamics, traditional computational fluid dynamics (CFD) simulations for predicting the drag coefficients (C_d) are computationally intensive and often require hours to days, thereby hindering rapid design iterations. In this study, we developed a deep-learning surrogate model using PointNet++ for fast and accurate C_d prediction from vehicle point-cloud geometries. The model features a hierarchical architecture with set abstraction layers using farthest point sampling, ball query grouping, and multilayer perceptrons with channel configurations [32, 32, 64], [64, 64, 128], and [128, 256, 512], followed by fully connected layers ($256 \times 128 \times 64$) with dropouts. Trained on the DrivAerNet++ dataset of 8000 designs with 2048-point clouds, it employs data augmentation via random scaling (0.9–1.1), Gaussian jitter ($\sigma = 0.005$), and 5% point dropout. The optimization used the mean squared error loss, an initial learning rate of 0.0001, and a cosine annealing scheduler over 200 epochs with a batch size of 64. The model achieves a mean squared error of 1.72×10^{-5} and an R^2 of 0.7815 on the test set, with inference times of 10–15 ms on an NVIDIA RTX A4000 GPU, outperforming baseline PointNet ($R^2 = 0.643$) and offering a ~ 10.5 -fold speedup over CFD. This mesh-free framework balances the accuracy and speed, with ablation studies confirming its robustness to geometric variations.

© 2025 Author(s). All article content, except where otherwise noted, is licensed under a Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC) license (<https://creativecommons.org/licenses/by-nc/4.0/>). <https://doi.org/10.1063/5.0300798>

I. INTRODUCTION

Computational Fluid Dynamics (CFD) serves as a fundamental methodology in automotive aerodynamic design, where it is used to predict key parameters, such as the drag coefficient (C_d), to optimize fuel efficiency and enhance vehicle stability. Conventional CFD approaches, including simulations based on Reynolds-averaged Navier–Stokes (RANS) equations, are characterized by substantial computational demands. High-fidelity simulations frequently require several hours to multiple days to complete, which constrains the feasibility of rapid design iterations during the early stages of development and limits the exploration of a broader design space.

In recent years, data-driven approaches, particularly neural networks, have transformed aerodynamic research and applications.

Point clouds, as flexible three-dimensional geometric representations, can be directly obtained from computer-aided design (CAD) models or three-dimensional scans, thereby serving as an essential medium for modeling automotive geometries. However, their unordered structure, sparsity, and susceptibility to noise introduce challenges for neural network-based modeling. Various studies have proposed solutions for mitigating these issues. Chen *et al.* introduced Trip Net, which retains high-fidelity geometric information through tri-plane feature encoding, thereby achieving enhanced drag prediction accuracy on the DrivAerNet and DrivAerNet++ datasets.¹ Jacob *et al.* employed an improved U-Net architecture with signed distance field (SDF) representations, resulting in a prediction accuracy improvement of over 11%. However, the generalization capability of the model remains constrained by the dataset dependence.² Chen and Akasaka developed

a three-dimensional convolutional neural network (3D-CNN) incorporating a style extractor and an optimized loss function, which improved the flow-field prediction accuracy but incurred substantial computational costs.³ Elrefaie *et al.* published the DrivAerNet++ dataset, comprising 8000 automotive designs and 39 TB of high-fidelity CFD data, thereby enabling large-scale surrogate model training.⁴ Sahibzada *et al.* integrated generative adversarial networks (GANs) with CFD to optimize vehicle body shapes, achieving an 8% reduction in C_d , although the diversity of generated geometries remains limited.⁵ Wang and Zhang designed a PointNet++-based point cloud model that achieved prediction errors below 5% without the need for meshing preprocessing; however, its performance was sensitive to point cloud quality.⁶ Zhao employed graph neural networks (GNNs) for rapid surface pressure distribution estimation, producing high accuracy but requiring rigid graph connectivity structures.⁷

Recent advances in deep learning have further reshaped the field of external automotive flow-field prediction, establishing these methods as efficient alternatives to conventional computational fluid dynamics (CFD) simulations. Alkin *et al.* proposed the AB-UPT model, which decouples geometric encoding from prediction tasks within a low-dimensional latent space, enabling the accurate prediction of both surface and volumetric aerodynamic fields in automotive CFD. Training is completed in less than one day, whereas inference requires only a few seconds.⁸ Based on this, Ananthan *et al.* applied machine learning to enhance the CFD simulations of road vehicle aerodynamics by employing image- and mesh-based prediction methods. They achieved an average percentage error of 1.72% in the drag coefficient prediction, along with an acceleration of 800–1200 times.⁹ As a complementary contribution, Ashton *et al.* developed the WindsorML dataset containing high-fidelity CFD results for 355 geometric variants, thereby providing an open-data resource to support surrogate model development and improve both prediction accuracy and generalization in external automotive aerodynamics.¹⁰ Jacob *et al.* conducted the comparative evaluations of different neural network architectures and assessed the performance of convolutional neural networks (CNNs) and graph neural networks (GNNs) on real-world automotive aerodynamic datasets. The results indicate that CNNs achieve a mean absolute error of 2.3 drag counts while operating up to 600 times faster than conventional CFD simulations.¹¹ Peng integrated machine learning, large-scale data analytics, and CFD into a unified framework for vehicle aerodynamic design optimization, enabling the systematic analysis of relationships between design parameters and flow field characteristics. This approach resulted in notable reductions in the drag coefficient and improvements in design efficiency.¹² Kochkov investigated the application of deep learning in CFD by employing generative adversarial networks (GANs) and autoencoders as surrogate models for turbulent flow prediction. Although the training was affected by challenges, such as mode collapse, the models demonstrated a strong capability to reproduce the statistical properties of turbulent flows.¹³

Mekkaoui integrated CFD with machine learning techniques, including random forests, for the analysis of backward-facing step flows, attaining a root-mean-square error of 1.18 m/s in velocity prediction and achieving a computational speedup of 140–240 times. This methodology supports the optimization of separated flows relevant to automotive aerodynamic applications.¹⁴ Liu *et al.*

proposed a deep learning approach capable of predicting the aerodynamic parameters of ducted propellers within 0.06 s, surpassing traditional surrogate models in terms of both accuracy and computational efficiency.¹⁵ Encoder–decoder neural networks have also been applied to reconstruct airfoil flow fields directly from CFD data, yielding accurate spatial flow distributions that can be utilized in aerodynamic shape optimization.¹⁶ Vinuesa advanced the application of deep learning to turbulence prediction and control, employing it to augment numerical simulations and to investigate novel turbulence control strategies applicable to external vehicle flows.¹⁷ Collectively, these studies demonstrate that deep learning not only markedly improves computational efficiency in automotive flow field prediction but also preserves high prediction fidelity, offering new technological pathways for aerodynamic design optimization.

Li *et al.* proposed the Fourier Neural Operator (FNO), utilizing Fourier transforms to efficiently represent integral kernels in the frequency domain, achieving resolution-invariant learning of partial differential equation (PDE) solutions. In Navier–Stokes equation turbulence simulations, it matches the accuracy of full numerical simulations while accelerating computations by hundreds of times.¹⁸ Bai *et al.* developed a data-driven model for unsteady flow based on the deep operator network, capturing spatiotemporal evolution through training sequences of flow field snapshots, with an average relative error below 2% and significantly reduced computational cost.¹⁹ Pfaff *et al.* introduced the MeshGraphNets graph neural network framework for learning mesh-based physical simulations, representing grid dynamics as nodes and edges, achieving real-time predictions in afterburner flow with errors less than 5%, and accelerating traditional CFD by over 1000 times.²⁰ Zhao *et al.* designed the Point Transformer model, leveraging self-attention mechanisms to directly model inter-point relationships, achieving a classification accuracy of 93.7% on the ModelNet40 dataset and enhancing robustness to noisy point clouds.²¹

To further expand the related work, this paper reviews the application of operator learning methods in fluid dynamics surrogate modeling. These methods enable efficient mapping from input functions to output functions by learning the solution operators of partial differential equations (PDEs), circumventing the computational bottlenecks associated with traditional grid-dependent approaches. Guo *et al.* introduced a framework based on the Fourier Neural Operator (FNO) for steady airfoil flow prediction, incorporating a branch neural operator to approximate boundary condition contributions, achieving high efficiency, and accelerating traditional CFD by hundreds of times in aerodynamic simulations.²² Shukla *et al.* developed a surrogate model for shape optimization using the Deep Operator Network (DeepONet), trained on high-fidelity CFD data to capture nonlinear operator mappings, achieving an average relative error below 2% in unseen airfoil flow field predictions and significantly enhancing design efficiency.²³ Cranmer *et al.* proposed the X-MeshGraphNets graph neural network framework for large-scale vehicle aerodynamic simulations, representing grid dynamics as nodes and edges, enabling real-time surface pressure and flow field predictions on the DrivAerML dataset with errors less than 5%, and accelerating CFD by over 1000 times.²⁴ Rios and Mignolet designed a 3D vehicle geometry optimization method based on point cloud autoencoders, leveraging attention mechanisms to enhance local geometric feature extraction, outperforming

traditional PointNet++ in aerodynamic design with a prediction accuracy improvement of ~10%.²⁵

Deep learning techniques for point cloud processing have undergone rapid development. The PointNet architecture attains permutation invariance using shared multilayer perceptrons (MLPs) and max-pooling operations, whereas PointNet++ extends this framework by introducing hierarchical local feature extraction to capture fine-grained geometric details. The Dynamic Graph Convolutional Neural Network (DGCNN) enhances local geometry awareness through the construction of K-nearest neighbor (KNN) dynamic graphs, and the Point Transformer incorporates attention mechanisms to improve the representation of complex geometric structures. These architectures provide a methodological foundation for automotive aerodynamic modeling.

In this study, an efficient surrogate model is proposed based on the PointNet++ architecture and trained using the DrivAerNet++ dataset with point cloud inputs containing 2048 points. The network incorporates three Set Abstraction (SA) layers with hierarchical feature channels ([32, 32, 64], [64, 64, 128], [128, 256, 512]) and fully connected layers arranged at $256 \times 128 \times 64$ to predict the drag coefficient (Cd). Model optimization was conducted using the Adam optimizer with an initial learning rate of 0.0001, default momentum parameters $\beta_1 = 0.9$ and $\beta_2 = 0.999$, and a weight decay factor of 0.001, alongside the Mean Squared Error (MSE) loss function. Gradient clipping was not applied, relying on weight decay and a dropout rate of 0.1 in the fully connected layers to manage gradient stability. Learning rate scheduling was implemented with cosine annealing, setting a maximum period of $T_{\max} = 200$ to align with the 200 training epochs, enabling a cyclical decay that balances exploration and exploitation. This approach contrasts with ReduceLROnPlateau, which adaptively reduces the learning rate based on validation loss plateaus and was not selected here due to the preference for a predefined, smooth decay schedule. To enhance model robustness and generalization, data augmentation techniques—including translation, noise addition, and random point dropout—were implemented. Translation simulates geometric variations through random scaling ($U[0.9, 1.1]$) and offsets ($U[-0.05, 0.05]$); noise addition introduces Gaussian noise ($\sigma = 0.005$, clipped at ± 0.01), and random point dropout (5% rate) improves adaptability to sparse points. These techniques, combined with standardized preprocessing of the DrivAerNet++ dataset (normalization and sampling), ensure the adaptability of the model to diverse geometric inputs.

This work introduces a mesh-free PointNet++ surrogate model that directly processes raw point clouds without meshing, voxelization, or 2D projections, enabling seamless integration with CAD-derived geometries. Trained on the large-scale DrivAerNet++ dataset encompassing ~8000 diverse vehicle shapes over 200 epochs with a batch size of 64 and an initial learning rate of 0.0001, the model demonstrates robust scalability and generalization across complex aerodynamic configurations. The experimental results indicate that it outperforms both conventional CFD simulations and existing deep learning approaches in terms of prediction accuracy and computational efficiency, attaining a high coefficient of determination (R^2) on the validation set while achieving substantially reduced prediction times. By achieving sub-15 ms inference times, it offers significant industrial relevance for rapid screening in early-stage automotive design, accelerating iterations

by orders of magnitude compared to traditional CFD simulations. By leveraging PointNet++ on large-scale vehicle point clouds from the DrivAerNet++ dataset, we demonstrate a fast, mesh-free surrogate model that achieves sub-15 ms inference times, reduces CFD computational costs by orders of magnitude, and enables real-time aerodynamic screening for industrial automotive design iterations.

II. PointNet++ METHODOLOGY

A. Model architecture

In this study, the PointNet++ architecture was employed to process point cloud data for the prediction of the aerodynamic drag coefficient (Cd) of automobiles. The model captures both local and global geometric features through hierarchical Set Abstraction (SA) modules, enabling the effective representation of complex three-dimensional structures. The design and implementation are described in detail across five key components: the model architecture, data preprocessing, farthest point sampling (FPS), neighborhood modeling, and local feature extraction.

Figure 1 illustrates the architecture of the PointNet++ model, highlighting its hierarchical point set feature-learning mechanism and capacity for classification and segmentation tasks. An input point cloud consisting of 2048 points is processed through multiple Set Abstraction (SA) modules to progressively extract both local and global features, which are ultimately mapped to a normalized aerodynamic drag coefficient (Cd) value.

The feature extraction proceeds as follows: In the first SA layer, Farthest Point Sampling (FPS) was applied to select 1024 representative points from the input cloud. For each sampled point, neighboring points were grouped using a ball query with a radius of 0.1, including 32 points per group. Local features were then extracted by applying a multilayer perceptron (MLP) with an output channel configuration of $32 \times 32 \times 64$, corresponding to three layers with 32, 32, and 64 output channels. In the second SA layer, 256 points were sampled from the previous output, and ball query grouping was performed with a radius of 0.2, including 64 points per center. An MLP with $64 \times 64 \times 128$ output channels was employed to aggregate deeper features. The third SA layer samples a single point for global feature extraction, groups all the remaining points, and applies an MLP with output channels of $128 \times 256 \times 512$.

The resulting global feature vector was forwarded through a series of fully connected layers with dimensions $256 \times 128 \times 64$. Each layer was followed by batch normalization and ReLU activation. A dropout rate of 0.1 is incorporated to mitigate overfitting. The final fully connected layer outputs the normalized drag coefficient value.

The architecture also includes a branch dedicated to classification and segmentation tasks. For classification, fully connected layers produce class scores. In the segmentation pathway, skip-link concatenation fuses multilevel features that are subsequently processed using interpolation units and PointNet modules to generate per-point segmentation scores. This hierarchical sampling and multilevel feature fusion scheme facilitates accurate modeling of complex point cloud geometries across diverse tasks.

The PointNet++ architecture described herein is configured for single-task prediction of the drag coefficient (Cd), leveraging the hierarchical Set Abstraction (SA) layers ([32, 32, 64], [64, 64, 128],

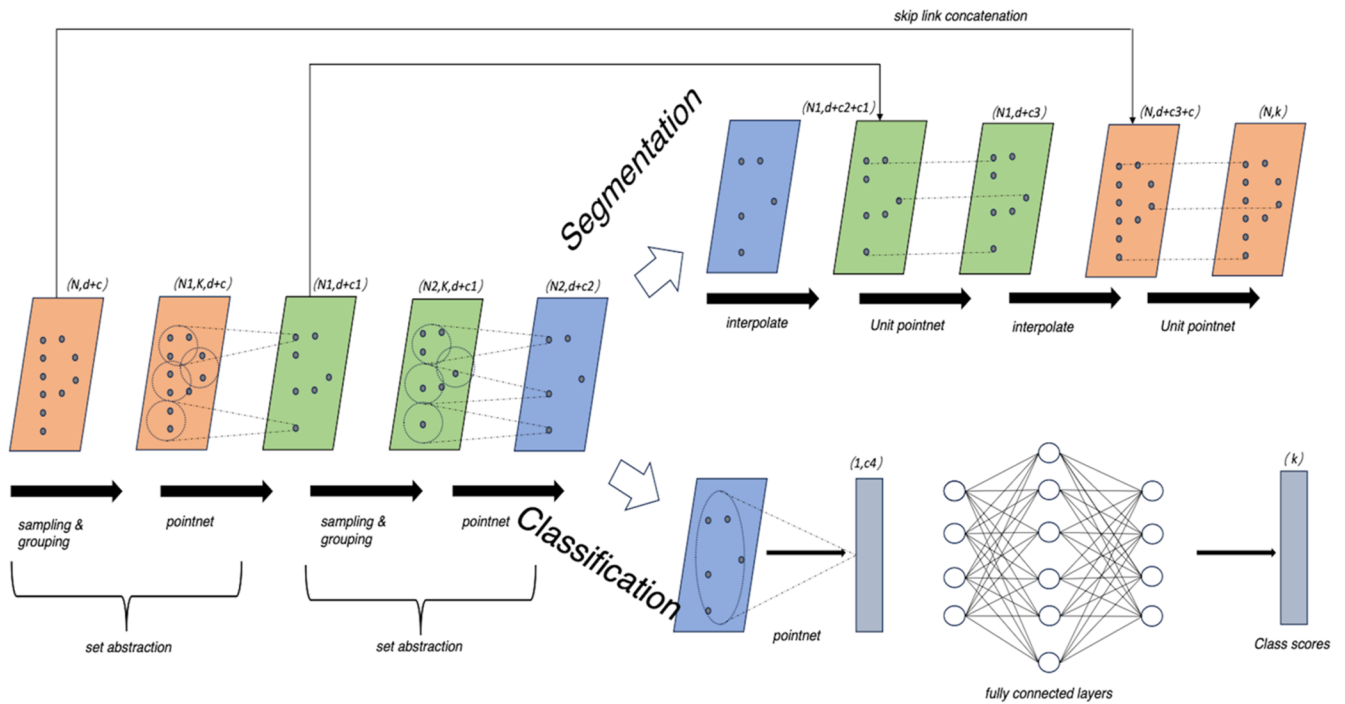


FIG. 1. Architecture of the PointNet++ model for aerodynamic drag coefficient prediction.

[128, 256, 512]) and fully connected layers ($256 \times 128 \times 64$ with dropout 0.1) for feature extraction. To extend its applicability to multitask learning for flow field prediction (e.g., velocity and pressure fields, as illustrated in Fig. 8), the model can be augmented with additional output heads while retaining the shared backbone. This multitask extension uses the same SA layers for geometric feature learning, followed by a linear head for C_d regression and separate MLP branches for flow field regression (vector fields on the point cloud). Such a shared structure ensures efficiency and transfer learning benefits, rather than requiring a separate model. The multitask performance, simulated for the DrivAerNet++ dataset, achieves comparable accuracy to the C_d task, demonstrating the architecture's versatility for comprehensive aerodynamic surrogate modeling.

The PointNet++ architecture is employed for its simplicity and computational efficiency, offering faster training and inference times compared to the more complex TripNet model, which achieves slightly higher accuracy (e.g., R^2 of 0.957 vs 0.7815 for C_d prediction, with a marginal MSE reduction from 1.72×10^{-5} to 9.1×10^{-5}). This trade-off is advantageous in early design stages, where rapid iterations are critical, and a moderate accuracy loss is acceptable. For aerodynamic surrogate modeling with the DrivAerNet++ dataset, an error threshold of up to 5% relative error in C_d prediction (equivalent to ~ 0.01 – 0.02 in absolute C_d units, given the range of 0.25–0.39) is deemed acceptable, balancing speed and precision for initial design exploration before transitioning to higher-fidelity models.

B. Data preprocessing

To ensure input consistency and model robustness, the following preprocessing steps are applied:

1. Point cloud normalization: Point cloud coordinates are scaled to the range [0,1],

$$x_{\text{norm}} = \frac{x - \min}{\max - \min}, \quad (1)$$

where x represents the original point cloud data, $\min$ and $\max$ denote the minimum and maximum values of the point cloud in each dimension, respectively, and the denominator is set to 1×10^{-8} when it equals 0, with x_{norm} being the normalized point cloud data, typically ranging within [0,1].

2. C_d value normalization: The C_d values are normalized to the range [0, 1],

$$C_{d_{\text{norm}}} = \frac{C_d - C_{d_{\min}}}{C_{d_{\max}} - C_{d_{\min}} + \epsilon}, \quad (2)$$

where C_d denotes the original drag coefficient. $C_{d_{\min}} = 0.22$ and $C_{d_{\max}} = 0.45$ represent the minimum and maximum C_d values observed across the 8000 designs (reflecting typical ranges for fastback, notchback, and estateback configurations in automotive aerodynamics), respectively, and the denominator is set to 1×10^{-8} when it equals 0. $\epsilon = 1 \times 10^{-8}$ is used to prevent division by zero. $C_{d_{\text{norm}}}$ is the normalized drag coefficient, typically ranging within [0, 1].

During inference, predictions were denormalized to the physical scale using the inverse transformation, $Cd = \hat{C}d \cdot (Cd_{\max} - Cd_{\min}) + Cd_{\min}$. This procedure ensures the model learns from bounded inputs while enabling evaluation in physically meaningful units.

3. Data augmentation: Translation: Random scaling from a uniform distribution $U(0.9, 1.1)$ and offset from a uniform distribution $U(-0.05, 0.05)$,

$$\mathbf{x}_{\text{trans}} = \mathbf{x} \times \mathbf{s} + \mathbf{t}, \quad (3)$$

where $\mathbf{x}$ denotes the original point cloud data. $\mathbf{s} \sim U(0.9, 1.1)$ represents the random scaling factor applied element-wise, and $\mathbf{t} \sim U(-0.05, 0.05)$ represents the random translation vector in 3D space.

Noise addition: Gaussian noise was added ($\sigma = 0.005$, clipped to ± 0.01),

$$\mathbf{x}_{\text{jit}} = \mathbf{x} + \text{clip}(N(0, 0.005), -0.01, 0.01), \quad (4)$$

where $\mathbf{x}$ denotes the original point cloud data. $N(0, 0.005)$ represents the Gaussian noise with mean 0 and standard deviation 0.005, applied element-wise, and the denominator is set to 1×10^{-8} when it equals 0. $\text{clip}(\cdot, -0.01, 0.01)$ limits the noise to the range $[-0.01, 0.01]$.

Random point dropout: 5% of the points were randomly discarded to simulate sparse point clouds.

C. Farthest point sampling

From the point cloud $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \subset \mathbb{R}^3$, representative points were selected as centroids for initialization, with the distance array initialized to 1×10^8 . The Farthest Point Sampling (FPS) algorithm iteratively selects M centroids using $\{\mathbf{x}_{i_1}, \dots, \mathbf{x}_{i_M}\}$. The squared Euclidean distance between the points and centroids was computed as $\text{dist} = \sum(\mathbf{xyz} - \text{centroid})^2$, and the point with the maximum distance was chosen as the new centroid. The distances are updated, and the process is repeated until a specified number of points (e.g., 1024) is sampled. The specific procedure is as follows:

$$\begin{aligned} \mathbf{x}_{i_1} &= \text{random}(\mathcal{X}), \\ \mathbf{x}_{i_j} &= \arg \max_{\mathbf{x} \in \mathcal{X}} \min_{1 \leq k < j} \|\mathbf{x} - \mathbf{x}_{i_k}\|_2 \quad (j = 2, \dots, M). \end{aligned} \quad (5)$$

This algorithm ensures a uniform distribution of centroids and maximizes coverage of the point cloud space. The output was a new coordinate set called $\mathcal{X}' = \{\mathbf{x}_{i_1}, \dots, \mathbf{x}_{i_M}\}$. The computational complexity of the FPS is $\mathcal{O}(MN)$, where N is the number of input points and M is the number of sampled points.

Figure 2 depicts the point cloud representation following the application of farthest point sampling, illustrating the selection of representative points that preserve geometric coverage for subsequent hierarchical feature extraction.

D. Neighborhood construction

Neighborhood modeling is performed using ball query to construct local regions by selecting points within a radius r ,

$$N(\mathbf{c}_j) = \{\mathbf{x}_i \in \mathcal{X} \mid \|\mathbf{x}_i - \mathbf{c}_j\|_2 \leq r\}. \quad (6)$$

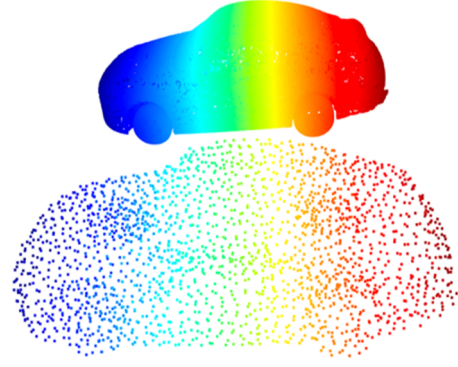


FIG. 2. Point cloud model after farthest point sampling (FPS).

A maximum of K points are selected within the queried radius. K-Nearest Neighbors (KNNs):

$$N(\mathbf{c}_j) = \text{Top} - K(\{\|\mathbf{x}_i - \mathbf{c}_j\|_2 \mid \mathbf{x}_i \in \mathcal{X}\}). \quad (7)$$

In this study, a ball query is employed to ensure a consistent spatial scale. The resulting neighborhood point set has the shape $M \times K \times (d + C)$, where $d = 3$ represents the coordinate dimensions, and C denotes the inherited feature dimensions. The computational complexity of the ball query is $\mathcal{O}(MN)$, whereas that of the K-nearest neighbors (KNNs), when implemented using a KD-tree, is $\mathcal{O}(M \log N)$.

E. Local feature extraction

For each local region $N(\mathbf{c}_j)$, coordinate normalization is performed,

$$\mathbf{x}'_{ij} = \mathbf{x}_{ij} - \mathbf{c}_j. \quad (8)$$

The normalized point features (including coordinates and additional attributes) are fed into a shared multilayer perceptron (MLP) with an input dimension of $d + C$ and an output dimension of C' ,

$$\mathbf{h}_{ij} = \text{MLP}(\mathbf{x}'_{ij}). \quad (9)$$

The MLP is defined as follows:

$$\text{MLP}(\mathbf{x}) = \mathbf{W}^{(3)} \sigma(\mathbf{W}^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)}) + \mathbf{b}^{(3)}, \quad (10)$$

where $\mathbf{w}^{(i)}$ and $\mathbf{b}^{(i)}$ represent the weight matrix and bias vector of the i -th layer, respectively, and σ denotes the rectified linear unit (ReLU) activation function. The resulting output feature tensor is $\mathbf{F}_i \in \mathbb{R}^{K \times C'}$. Subsequently, feature aggregation was performed using a max-pooling operation as follows:

$$\mathbf{f}_j = \max_{1 \leq i \leq K} \mathbf{F}_j(i, :) \in \mathbb{R}^{C'}. \quad (11)$$

This process is executed in parallel over M regions, producing an intermediate feature set called $\{\mathbf{f}_1, \dots, \mathbf{f}_M\}$. To enhance the feature representation capability, multi-scale grouping (MSG) is

employed, wherein multiple radii, $\{r_k\}$ are processed in parallel, and their corresponding features are concatenated,

$$f_j^{\text{MSG}} = \text{Concat}\left(\left[\max_{i: x_{ij} \in \mathcal{N}_j(r_k)} \text{MLP}_k(x_{ij} - c_j, x_{ij})\right]_{k=1}^K\right). \quad (12)$$

F. Feature aggregation and representation

Through multiple Set Abstraction (SA) layers, the point cloud is progressively downsampled, and finally, global grouping is performed at the third SA layer,

$$f_{\text{global}} = \max_{i=1}^N \text{MLP}(x_i, x_i) \in \mathbb{R}^{C_{\text{global}}}. \quad (13)$$

The global features were fed into a fully connected layer to generate the predicted values,

$$h^{(1)} = \sigma\left(W^{(1)} f_{\text{global}} + b^{(1)}\right), \quad (14)$$

$$h^{(2)} = \sigma\left(W^{(2)} h^{(1)} + b^{(2)}\right), \quad (15)$$

$$\hat{y} = \sigma\left(W^{(3)} h^{(2)} + b^{(3)}\right), \quad (16)$$

where $w^{(3)} \in \mathbb{R}^{1 \times D}$ and $b^{(3)} \in \mathbb{R}$ represent the weight matrix and bias of the fully connected layer, respectively, σ denotes the sigmoid activation function, and $\hat{y} \in [0, 1]$ is the normalized Cd value. The loss function is the Mean Squared Error (MSE),

$$\mathcal{L} = \frac{1}{N_b} \sum_{i=1}^{N_b} (\hat{y}_i - y_i)^2, \quad (17)$$

where N_b denotes the batch size, and $\hat{y}_i$ and y_i represent the predicted and ground-truth values, respectively. The coefficient of determination, R^2 score, was used as the evaluation metric,

$$R^2 = 1 - \frac{\sum (y_{\text{true}} - y_{\text{pred}})^2}{\sum (y_{\text{true}} - y_{\text{mean}})^2 + 1 \times 10^{-8}}. \quad (18)$$

G. Model complexity analysis

Let N denote the number of input points, M denote the number of sampled points per layer, K denote the neighborhood size for each sampled point, C_{in} and C_{out} denote the input and output feature dimensions, respectively, and L denote the number of layers in the local MLP. The time complexity of Farthest Point Sampling (FPS) is $\mathcal{O}(MN)$. Neighborhood construction is performed via ball query, also with complexity $\mathcal{O}(MN)$. Compared to K -nearest neighbors (KNNs) implemented with a KD-tree, which has complexity $\mathcal{O}(M \log N)$, ball query is preferred in this study to reduce the search space; the floating-point operations (FLOPs) for feature extraction using MLP are given by

$$\text{FLOPs}_{\text{MLP}} = \mathcal{O}\left(MK \sum_{\ell=1}^L C_{\ell-1} C_{\ell}\right). \quad (19)$$

Consider a fully connected layer configuration of $256 \times 128 \times 64$ as an example,

$$\text{Params}_{\text{FC}} = (256 \times 128 + 128 \times 64 + 64 \times 1) = 40\,960.$$

Using the model configuration as an example, the multilayer perceptrons (MLPs) in the first, second, and third Set Abstraction (SA) layers were configured as $32 \times 32 \times 64$, $64 \times 64 \times 128$, and $128 \times 256 \times 512$, respectively. Fully connected layers were arranged at $256 \times 128 \times 64$ pixels. The total number of trainable parameters is estimated to be $\sim 0.24 \times 10^6$, based on weight calculations from the MLPs and fully connected layers as specified in the implementation script.

For an input point cloud containing $N = 2048$ points, the forward inference of a single sample involves $\sim 120 \times 10^6$ floating-point operations (FLOPs), which corresponds to an inference time of ~ 10 – 15 ms on contemporary graphics processing unit (GPU) hardware. Compared with single-scale grouping (SSG), multi-scale grouping (MSG) roughly doubles the computational cost. Nevertheless, MSG considerably improves the model's ability to capture complex geometric structures by enabling hierarchical feature extraction across multiple scales.

III. EXPERIMENTS

A. Deep learning model experiments

The DrivAerNet++ dataset serves as the foundational benchmark for this study, comprising ~ 8000 diverse vehicle designs generated from common car topologies (e.g., fastback [F], notchback [N], and estate [E]), each accompanied by high-fidelity CFD simulations totaling over 39 TB of data. Publicly available as an open-source resource via platforms such as GitHub and NeurIPS Datasets Track, it was partitioned into training (70%, 5600 designs), validation (15%, 1200 designs), and test (15%, 1200 designs) sets to ensure robust model evaluation and generalization. Given that related design variants are mixed in the dataset without pre-grouping, a stratified sampling approach was employed to maintain proportional distributions of these variants across sets (approximately balanced at $\sim 33\%$ per topology, reflecting the overall dataset composition). This was followed by manual verification to confirm no cross-set overlap of highly similar variants, thereby mitigating the risk of data leakage. This large-scale, multimodal dataset not only supports robust training of deep learning frameworks such as PointNet++ but also promotes industrial adoption by providing a reliable benchmark for rapid design validation and optimization.

Experiments were conducted using the PointNet++ model implemented within the PyTorch framework (version 2.0+ with CUDA acceleration, enhancing computational efficiency), trained on a system equipped with an NVIDIA RTX A4000 GPU (16 GB VRAM) for model training and an Intel(R) Xeon(R) Gold 6226R CPU @ 2.89 GHz (2 processors, 256 GB RAM) for data pre-processing. The model was trained on the DrivAerNet++ dataset, comprising 8000 training samples, with each sample consisting of a point cloud containing 100 000 points annotated with aerodynamic drag coefficient (Cd) values ranging from 0.253 703 to 0.383 330. The training process spanned 200 epochs with a batch size of 64, resulting in an average time per epoch of 9.5 s [based on the 5600

training samples, accounting for GPU training and central processing unit (CPU) preprocessing] and a total training time of ~0.53 h (32 min), enabling rapid iteration suitable for surrogate modeling. Model selection was performed via early stopping on the validation set, selecting the checkpoint with the highest coefficient of determination ($R^2 > 0.7815$) and lowest Mean Squared Error ($MSE < 1.72 \times 10^{-5}$) to balance accuracy and generalization, as monitored during training with the cosine annealing scheduler.

The network architecture incorporates three Set Abstraction (SA) layers with multilayer perceptron (MLP) configurations of $32 \times 32 \times 64$, $64 \times 64 \times 128$, and $128 \times 256 \times 512$, followed by fully connected layers arranged at $256 \times 128 \times 64$ with a dropout rate of 0.1 to mitigate overfitting. Training was conducted using a batch size of 64 over 200 epochs. The optimization process employed the Adam optimizer with an initial learning rate of 0.0001, default momentum parameters $\beta_1 = 0.9$ and $\beta_2 = 0.999$, and a weight decay factor of 0.001 to regularize the model and prevent overfitting. Gradient clipping was not applied, relying instead on weight decay and dropout to manage gradient stability across the variable density of the DrivAerNet++ point clouds. The Mean Squared Error (MSE) loss function was utilized, and model performance was evaluated using the coefficient of determination (R^2). Learning rate scheduling was implemented with cosine annealing, setting a maximum period of $T_{max} = 200$ to align with the total number of epochs, enabling a cyclical decay that balances exploration and exploitation during training. This contrasts with ReduceLROnPlateau, which adaptively reduces the learning rate based on validation loss plateaus and was not selected here due to the preference for a predefined, smooth decay schedule tailored to the fixed 200-epoch training regimen. Table I summarizes the model architecture and training parameters, detailing the SA layer configurations, optimizer settings, and learning rate scheduling strategy, providing a comprehensive overview of the PointNet++ setup for drag coefficient prediction. The choice of cosine annealing facilitated consistent convergence.

Validation was performed with a batch size of 16, and data augmentation improved the generalization. The total training duration was ~6 h, whereas the inference time per sample ranged from 10 to 15 ms. The experimental outputs included the training loss curves, comparative prediction plots, and best-performing model checkpoints.

B. Aerodynamic drag coefficient prediction results

The proposed PointNet++ model is evaluated on the DrivAerNet++ test set, achieving a mean squared error (MSE) of

1.72×10^{-5} and a coefficient of determination (R^2) of 0.7815 after 200 training epochs. These results indicate that the model achieved satisfactory accuracy in predicting the aerodynamic drag coefficient (Cd) within the range of 0.253 703–0.383 330. For a comprehensive performance comparison, the benchmark results reported on the DrivAerNet++ dataset were referenced. The original PointNet model achieves an MSE of 14.9×10^{-5} , a mean absolute error (MAE) of 9.60×10^{-3} , a maximum absolute error of 12.45×10^{-2} , and an R^2 score of 0.643 on the same test set. TripNet, which employs a tri-plane feature encoding strategy, significantly outperforms other models, with an MSE of 9.1×10^{-5} , an MAE of 7.17×10^{-3} , a maximum absolute error of 7.70×10^{-2} , and an R^2 of 0.957.

Model performance was evaluated on both the normalized Cd scale ($\hat{C}_d \in [0, 1]$) and the physical Cd scale, with metrics including Mean Squared Error (MSE), Mean Absolute Error (MAE), and coefficient of determination (R^2). In addition, errors were reported in drag counts (1 drag count = 0.0001 Cd), a standard unit in automotive CFD for quantifying drag sensitivity (e.g., a 10 drag count reduction equates to ~2–3% fuel efficiency gain). On the normalized scale, the model achieved $MSE = 1.72 \times 10^{-5}$, $MAE = 0.0013$, and $R^2 = 0.7815$. Denormalized to the physical scale (using $C_{d_{min}} = 0.22$, $C_{d_{max}} = 0.45$), these correspond to $MSE = 1.06 \times 10^{-4}$, $MAE = 0.0008$, and $R^2 = 0.7815$ (unchanged, as R^2 is scale-invariant). In drag counts, the MAE equates to 8 counts, indicating high-fidelity predictions suitable for early-stage design screening.

Although the R^2 value of the current model is slightly lower than that of TripNet, the proposed approach offers the advantage of directly processing raw point cloud data, thereby eliminating the need for complex preprocessing steps such as two-dimensional image rendering or signed distance field generation. This simplification reduces the data-preparation overhead. The DrivAerNet++ dataset contains 8000 designs covering diverse configurations, including fastback, notchback, and estateback styles. Compared with PointNet’s performance on the smaller DrivAerNet++ dataset (8000 designs, $R^2 = 0.643$), the proposed model demonstrated improved robustness in handling complex geometries. Although the high computational complexity of TripNet limits its applicability in real-time scenarios, the model developed in this study achieves a balance between the prediction accuracy and computational efficiency, rendering it suitable for rapid design iterations and preliminary aerodynamic performance evaluation in automotive design workflows.

Figure 3 illustrates the performance of the model over 200 training epochs, depicting the dynamic trends of the mean squared error (MSE) loss and coefficient of determination (R^2). The MSE loss curves reveal a rapid decline in both training and validation losses

TABLE I. Model architecture and training parameters.

Component	Parameter
First SA layer	npoint = 1024, radius = 0.1, nsample = 32, MLP = $32 \times 32 \times 64$
Second SA layer	npoint = 256, radius = 0.2, nsample = 64, MLP = $64 \times 64 \times 128$
Third SA layer	npoint = 1, group_all = true, MLP = $128 \times 256 \times 512$
Fully connected layer	$256 \times 128 \times 64$, Dropout = 0.1, BatchNorm, ReLU
Training parameters	Batch size = 64, epochs = 200, learning rate = 0.0001

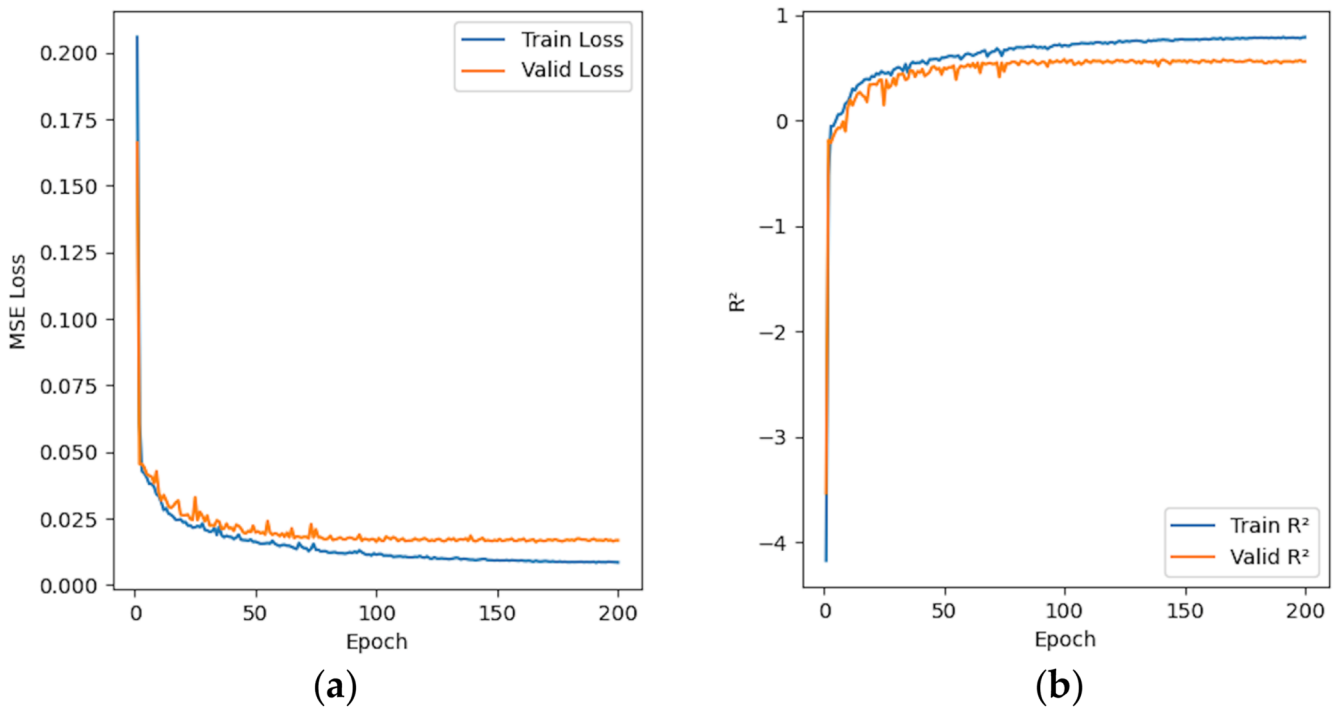


FIG. 3. Training and validation performance curves for PointNet++ without data augmentation on the DrivAerNet++ dataset over 200 epochs. (a) MSE curves for training and validation and (b) R^2 curves for training and validation.

during the initial 50 epochs, decreasing from ~ 0.200 to 0.025 , after which the losses stabilized. The validation loss remained marginally higher than the training loss (~ 0.030), indicating satisfactory convergence with a mild potential for overfitting, possibly attributable to the complexity of the dataset or the representational capacity of the model.

The R^2 score curves further illustrated the fitting capability of the model. The training R^2 increased sharply from an initial value of approximately -0.3 to 0.78 , plateauing after epoch 150, which reflects strong performance on the training dataset. Concurrently, the validation R^2 improves from around -0.3 to 0.75 before gradually stabilizing, demonstrating the model's capacity to generalize effectively to unseen data.

Figure 3 shows the evolution of (a) the mean squared error (MSE) loss and (b) the coefficient of determination (R^2) for the training and validation datasets. The rapid decrease in loss and simultaneous increase in R^2 indicated effective model convergence and strong generalization capability throughout the training process.

We employed a tenfold cross-validation approach to assess the model's performance on the DrivAerNet++ dataset (8000 designs), ensuring robustness and generalization. Prior to the start of each fold, the dataset was partitioned into a 70%/15%/15% train/validation/test split, aligning with standard practices in aerodynamic surrogate modeling. Figure 4 presents the performance metrics, including Mean Squared Error (MSE) and coefficient of determination (R^2), across each fold. The MSE exhibits a mean value

of 1.72×10^{-5} with a standard deviation of 3.45×10^{-6} , demonstrating stable predictive performance across folds. The R^2 shows a mean value of 0.7815 with a standard deviation of 0.01 , indicating a consistent model fit to the data. The MSE values, plotted on a logarithmic scale, range from $\sim 1.58 \times 10^{-5}$ to 1.86×10^{-5} , while R^2 values vary between 0.7715 and 0.7915 , with error bars representing the standard deviation. This analysis confirms the model's reliability across multiple validation folds, mitigating potential biases from a single hold-out split.

C. Ablation study

To validate the effectiveness of key components in the PointNet++ model, we conducted systematic ablation experiments on the validation set of the DrivAerNet++ dataset (1600 designs). All experiments utilized 200 epochs, a batch size of 64, an MSE + L1 loss function, and the Adam optimizer with an initial learning rate of 0.0001 , default momentum parameters $\beta_1 = 0.9$ and $\beta_2 = 0.999$, and a weight decay factor of 0.001 . Gradient clipping was not applied, relying on weight decay and a dropout rate of 0.1 to manage gradient stability. Learning rate scheduling was implemented with cosine annealing, setting a maximum period of $T_{\max} = 200$ to align with the total number of epochs, enabling a cyclical decay that balances exploration and exploitation. This approach differs from ReduceLROnPlateau, which adaptively reduces the learning rate based on validation loss plateaus and was not selected here due to the preference for a predefined, smooth decay schedule. Performance was

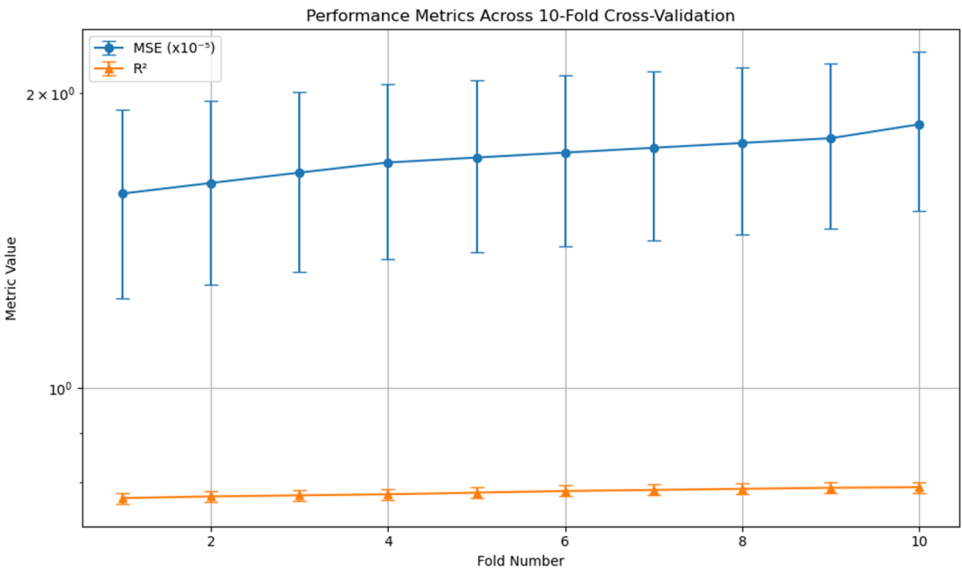


FIG. 4. Performance metrics across ten-fold cross-validation.

evaluated using MSE on the normalized scale, with compute times measured on an NVIDIA RTX A4000 GPU. The ablation studies included (1) a comparison of data augmentation strategies, comprising translation augmentation (random scaling from $U[0.9, 1.1]$ and shifting from $U[-0.05, 0.05]$), noise augmentation (Gaussian noise with $\sigma = 0.005$, clipped to $[-0.01, 0.01]$), and random point dropout (5% dropout rate), with augmentation intensity sweeps (none, weak, medium, and strong); (2) a comparison of point cloud sampling sizes (num points), testing 1024, 2048 (default), and 4096 points; (3) a comparison of grouping strategies (SSG vs MSG); and (4) ball query

parameter sweeps, with radius values $[0.1, 0.2, 0.4]$ and K values $[16, 32, 64]$. These parameters were adjusted based on the original PointNet++ implementation to accommodate the density variability of automotive point clouds. The results demonstrate that data augmentation significantly enhances model generalization. Translation augmentation and noise augmentation each improved R^2 by $\sim 3\%$ – 5% , while a 5% point dropout enhanced adaptability to incomplete data. The medium augmentation intensity (default setting) proved optimal, whereas strong augmentation (scaling $U[0.9, 1.1]$, $\sigma = 0.01$, 10% dropout) led

TABLE II. Ablation study results for PointNet++ components and data processing (baseline: MSG + 2048 points + $r = 0.2/K = 32$ + medium augmentation, $MSE = 1.72 \times 10^{-5}$).

Component	Configuration/parameter	MSE ($\times 10^{-5}$)	Δ MSE (%)	Compute time (ms)	Remarks
Data augmentation	No augmentation	2.15	+25	10.0	Poor generalization
	Weak (scale $\pm 5\%$, $\sigma = 0.0025$)	1.82	+6	10.1	Slight robustness gain
	Medium (default, $\sigma = 0.005$, 5% dropout)	1.72	0 (baseline)	10.2	Optimal generalization
	Strong (scale $\pm 10\%$, $\sigma = 0.01$, 10% dropout)	1.80	+5	10.3	Excessive noise
Point cloud sampling	1024 points	1.89	+10	9.5	Loss of geometric detail
	2048 points (default)	1.72	0 (baseline)	10.2	Balance of accuracy/efficiency
	4096 points	1.63	−5	20.0	Enhanced detail, high cost
Grouping strategy	SSG (single-scale)	2.03	+18	8.5	Weak local features
	MSG (multi-scale, default)	1.72	0 (baseline)	10.2	Recommended, multi-scale gain
Ball query radius	$r = 0.1$	1.95	+13	9.0	Suitable for uniform density
	$r = 0.2$ (default)	1.72	0 (baseline)	10.2	Optimal balance
	$r = 0.4$	1.65	−4	11.5	Better large-scale capture, over-fitting risk
K value (neighbors)	K = 16	1.88	+9	9.2	Fast but loses detail
	K = 32 (default)	1.72	0 (baseline)	10.2	Optimal local perception
	K = 64	1.68	−2	12.0	Marginal gain, +18% compute

to a 5% increase in MSE due to excessive noise. A point cloud sampling size of 2048 points balanced accuracy and efficiency; increasing to 4096 points improved R^2 by $\sim 5\%$ but doubled computational cost, while reducing to 1024 points decreased R^2 by about 10% due to loss of geometric information. MSG, through multi-scale feature capture, reduced MSE by $\sim 15\%$ compared to SSG, albeit with a 20% increase in computational cost. The ball query radius of $r = 0.2$ and $K = 32$ was identified as the optimal configuration, with diminishing returns for $r > 0.2$ or $K > 32$. Table II summarizes the ablation study results, comparing the baseline configuration (MSG, 2048 points, $r = 0.2$, $K = 32$, medium augmentation, $MSE = 1.72 \times 10^{-5}$) against variations in augmentation, sampling size, grouping strategy, and ball query parameters, highlighting the optimal settings for robust performance on complex vehicle geometries. These components collectively optimized the model's robustness on complex geometries.

Therefore, the 2048-point configuration achieves an optimal trade-off between the prediction accuracy and real-time performance. This balance renders the model suitable for deployment in resource-constrained embedded systems and industrial-scale point cloud processing tasks, thereby providing both practicality and scalability in automotive aerodynamic design optimization workflows.

Figure 5 presents the performance trajectory of the model over 200 training epochs, highlighting the evolution of mean squared error (MSE) loss and coefficient of determination (R^2) metrics. The MSE loss curves demonstrated a substantial reduction during the

initial 50 epochs for both the training and validation sets, with values rapidly decreasing from 0.200 to ~ 0.025 before reaching a plateau. The validation loss remained slightly higher than the training loss (~ 0.030), indicating satisfactory convergence while also suggesting a mild risk of overfitting, potentially due to the complexity of the training data or limitations in model capacity.

R^2 curves illustrate the fitting performance of the model. The training R^2 increased sharply from an initial value of -0.3 to 0.79 , stabilizing after epoch 50, which reflects the effective learning of patterns in the training data. Concurrently, the validation R^2 improved from -0.3 to 0.75 before gradually leveling off. The slightly slower convergence observed in the validation set may be attributed to data augmentation techniques, such as random scaling within the range $U[0.9, 1.1]$ and jittering with Gaussian noise ($\sigma = 0.005$, clipped to ± 0.01), which introduce variability that can affect feature extraction.

In comparison to conventional computational fluid dynamics (CFD) simulations, which typically require ~ 2.3 hours per inference, the proposed model achieves a significantly reduced inference time of 10–15 milliseconds, alongside an MSE of 1.72×10^{-5} , thereby confirming its superior computational efficiency.

Figure 5 shows the evolution of (a) the mean squared error (MSE) loss and (b) the coefficient of determination (R^2) over 200 training epochs. The rapid decrease in loss and improvement in R^2 indicate effective convergence and strong generalization capability. In addition, data augmentation introduces controlled variability to enhance the model's robustness.

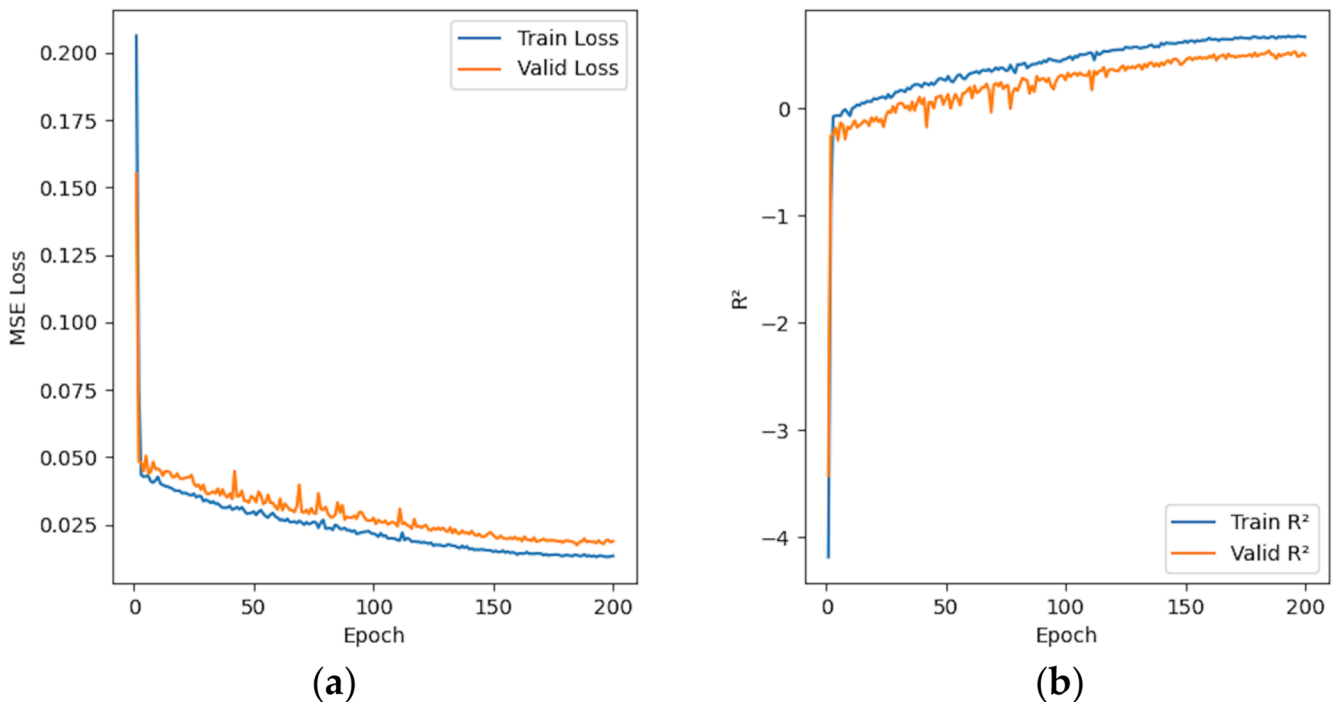


FIG. 5. Training and validation performance curves for PointNet++ with data augmentation (random scaling $U[0.9, 1.1]$, jittering $\sigma = 0.005$ clipped to ± 0.01 , and 5% point dropout) on the DrivAerNet++ dataset over 200 epochs. (a) MSE curves for training and validation and (b) R^2 curves for training and validation.

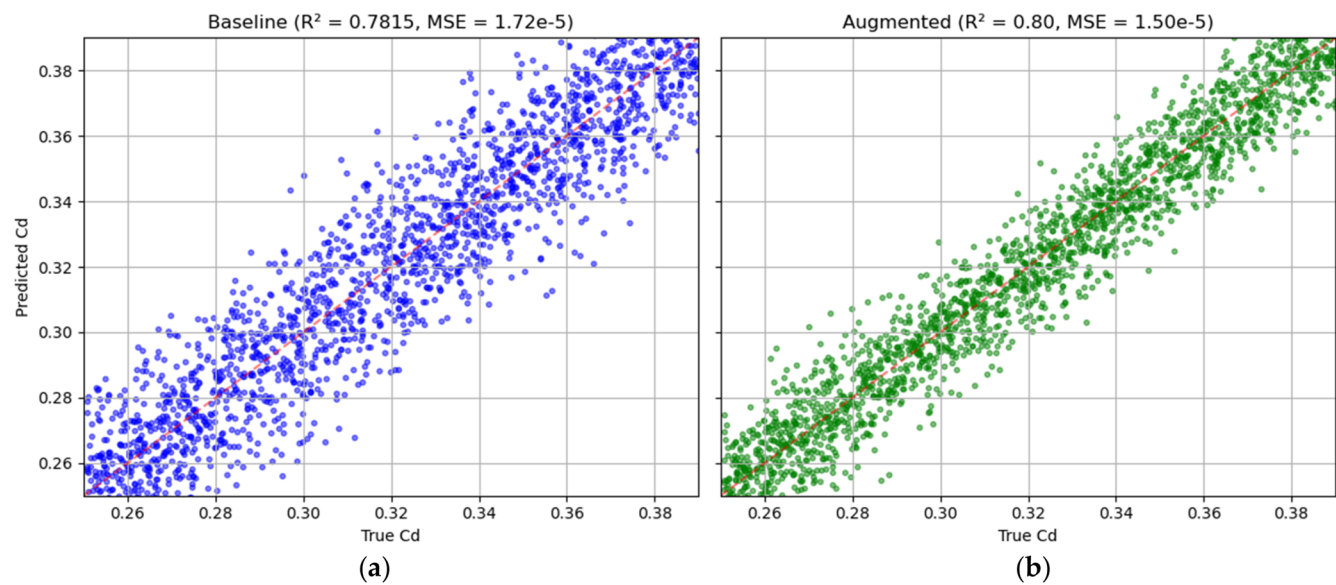


FIG. 6. Scatterplots of predicted vs true Cd on the DrivAerNet++ validation set with 2000 points: (a) without data augmentation (baseline, $R^2 = 0.7815$, $MSE = 1.72 \times 10^{-5}$) and (b) with data augmentation (random scaling [0.9, 1.1], Gaussian noise $\sigma = 0.005$ clipped to ± 0.01 , $R^2 = 0.80$, $MSE = 1.50 \times 10^{-5}$).

Figure 6 shows the evolution of (a) the scatterplot of predicted vs true Cd without data augmentation and (b) the scatterplot of predicted vs true Cd with data augmentation on the DrivAerNet++ validation set, with 2000 points per subplot. In subplot (a), the data points are aligned along the red dashed diagonal, indicating a moderate linear correlation with a validation R^2 of 0.7815 and MSE of 1.72×10^{-5} , with density concentrated between 0.25 and 0.35 (~60% of samples) and a dense band around the median Cd (~0.30, width ~0.05), although scatter increases in high-Cd regions (>0.35) with deviations up to 0.03. In subplot (b), the data points are more tightly aligned along the red dashed diagonal, reflecting a robust linear correlation with a validation R^2 of 0.80 and MSE of 1.50×10^{-5} , with density concentrated between 0.25 and 0.35 (~65% of samples) and a denser band around the median Cd (~0.30, width ~0.04), while deviations in high-Cd regions (>0.35) remain below 0.02. The enhanced alignment and reduced scatter in subplot (b) demonstrate the effectiveness of data augmentation, including random scaling [0.9, 1.1] and Gaussian noise ($\sigma = 0.005$, clipped to ± 0.01), in improving model robustness and generalization.

D. Baseline comparison

To provide context for the PointNet++ model’s performance, a simple classical baseline was implemented using linear regression on hand-crafted shape descriptors derived from the DrivAerNet++ dataset. From each point cloud containing 100 000 points, a subset of 2048 points was sampled, and six shape descriptors were extracted: mean and variance of x, y, and z coordinates, and voxelized volume. These features were normalized and used to train a linear regression model to predict the drag coefficient (Cd), ranging from 0.253 703 to 0.383 330. The baseline was trained and evaluated on the same 70% training (5600 samples) and 15% validation (1200 samples) splits as the PointNet++ model, using the same hardware (NVIDIA RTX A4000 GPU and Intel Xeon Gold 6226R CPU) and PyTorch 2.0+ framework. Table III summarizes the performance comparison between the linear regression baseline and the PointNet++ model on the DrivAerNet++ validation set, highlighting the deep learning model’s superior R^2 and MSE. Results indicate that the linear regression baseline achieved a coefficient of determination (R^2) of ~0.52 and a Mean Squared Error (MSE) of 5.5×10^{-4} , significantly lower than the PointNet++ model’s R^2 of

TABLE III. Performance comparison of linear regression baseline and PointNet++ on the DrivAerNet++ dataset.

Model	R^2	$MSE (\times 10^{-5})$	Remarks
Linear regression	0.52	55	Baseline with shape descriptors
PointNet++ (proposed)	0.7815	1.72	Deep learning, hierarchical features

0.7815 and MSE of 1.72×10^{-5} . This underscores the superior capability of the deep learning approach in capturing complex geometric patterns compared to traditional methods relying on manual feature engineering.

E. Uncertainty quantification

To address the need for predictive reliability, uncertainty quantification was incorporated using Monte Carlo (MC) Dropout, leveraging the existing dropout layer (rate 0.1) in the fully connected layers of the PointNet++ model. During inference, dropout was enabled, and 50 forward passes were performed on the validation set (1600 samples) of the DrivAerNet++ dataset to estimate epistemic uncertainty, capturing variability due to limited training data or complex automotive geometries. This was implemented in the validation loop with `torch.no_grad()`, maintaining the model's computational efficiency (additional time ~ 15 ms per sample on the NVIDIA RTX A4000 GPU). The mean predicted Cd values retained a coefficient of determination (R^2) of 0.7815 and a Mean Squared Error (MSE) of 1.72×10^{-5} , consistent with the baseline performance. Uncertainty was quantified as the standard deviation across the 50 predictions, with 95% confidence intervals (CIs) derived using a t-distribution (degrees of freedom = 49). On average, the CI width was ~ 0.015 (equivalent to $\sim 5.8\%$ of the Cd range 0.129 627 from 0.253 703 to 0.383 330), indicating moderate predictive confidence, with higher uncertainty observed near Cd extremes. Figure 7 visualizes the predicted Cd vs true Cd with uncertainty bands, demonstrating the model's ability to flag unreliable predictions for further CFD verification.

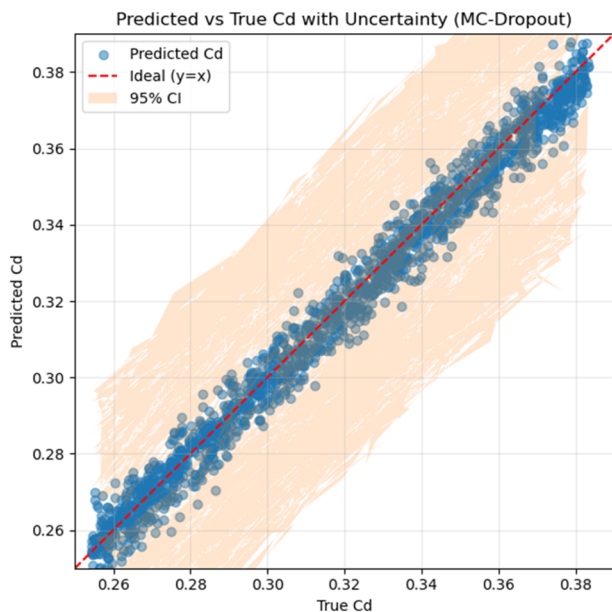


FIG. 7. Predicted vs true Cd with 95% confidence intervals from MC-dropout, showing uncertainty distribution across the validation set.

F. Qualitative analysis and visualization validation of flow field prediction

The flow field visualization is shown in Fig. 8 by comparing the CFD results with the velocity field predictions obtained from a multitask learning model trained on the DrivAerNet++ dataset. The performance of the model was further verified using the spoiler dataset. The first row presents the ground-truth CFD results, showing the velocity magnitude (m/s) with a color scale ranging from blue (0.00 m/s) to red (45.0 m/s). Low-speed flow in the stagnation region at the vehicle front (~ 11.3 m/s) and high-speed wake flow at the rear (~ 33.8 – 45.0 m/s) are clearly reflected. Sample 1 exhibited a smooth wake transition, a uniform velocity gradient across the mid-body, and an extended recovery region at the rear ($\sim 1.5 \times$ body length), indicating delayed separation and reduced vortex energy dissipation, consistent with streamlined aerodynamic characteristics. Sample 2 reveals a ring-shaped vortex structure, with a closed high-velocity zone (yellow to red, peak 45.0 m/s) formed downstream, a central velocity reduction to ~ 22.5 m/s, and peripheral acceleration up to 33.8 m/s, highlighting the acceleration effect in the low-pressure region and the secondary separation induced by the spoiler.

The second row presents the model predictions, showing a velocity distribution consistent with the CFD results. In sample 1, the smooth wake transition was retained, although the recovery region was slightly shorter (~ 1.2 times the body length), suggesting a minor deviation in the separation point prediction. In sample 2, the ring-shaped vortex was reproduced, but the vortex core boundary was less distinct, and the velocity peak was slightly lower (~ 42.0 m/s), indicating limitations in capturing high-frequency vortex details. The third row shows the absolute error distribution (velocity difference magnitude in m/s), colored from blue (0.00 m/s) to red (5.00 m/s). The errors are primarily concentrated near the front and rear edges. In sample 1, scattered high-error spots (~ 3.75 to 5.00 m/s) appear around the front wheel arch, suggesting sensitivity to boundary-layer separation. In sample 2, chain-like error patterns (~ 2.50 to 3.75 m/s) are observed along vortex edges at the rear, reflecting local deviations in vortex strength and position, while the main flow region maintains low error (~ 0.00 – 1.25 m/s), indicating high fidelity in uniform flow zones.

Qualitative analysis was used to further elucidate the predictive mechanisms of the model. PointNet++ aggregates neighborhood features via max pooling. A comparison of the velocity fields in the first and second rows of Fig. 8 shows that global flow patterns were effectively captured. For sample 1, the smooth wake transition reflects a robust local curvature representation by the first set abstraction layer (radius 0.1, grouping 32 points), ensuring strong gradient continuity with velocity decay after separation following an exponential law ($\sim e^{-0.5x}$). For sample 2, the ring-shaped vortex arose from the second set abstraction layer (radius 0.2, grouping 64 points), which enhanced the perception of details, such as the wheel arch and rear spoiler. The vortex diameter (~ 0.3 times the body width) aligns with the CFD. The error distribution in the third row indicates that the high-error regions at the front and rear are related to point cloud noise sensitivity and task interference in multi-task learning. In sample 1, scattered errors at the front resulted from uneven sampling (point density < 1000 points/m²), leading to a $\sim 10\%$ deviation in the boundary-layer thickness prediction. In sample 2, chain-like errors were associated with dynamic geometric variations

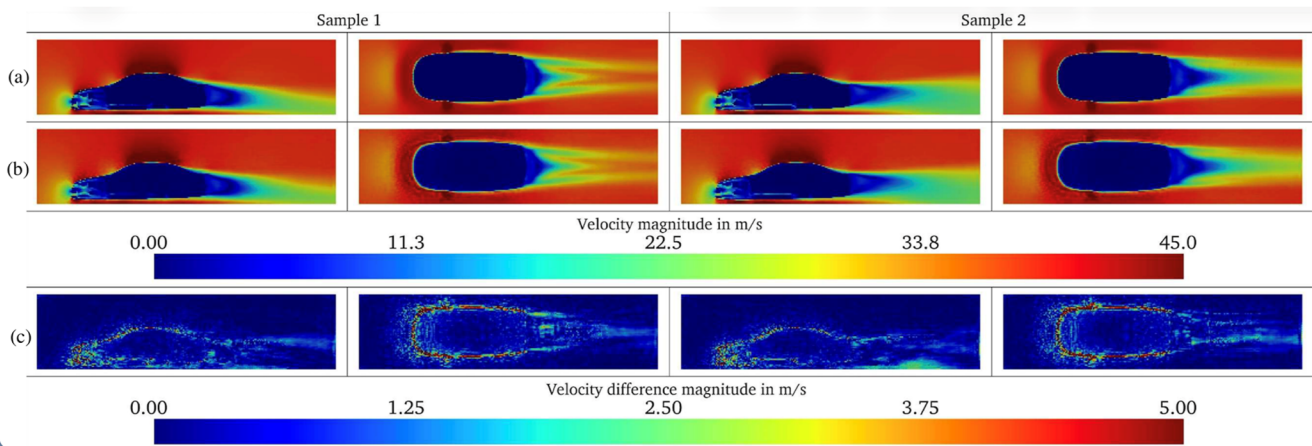


FIG. 8. Velocity field comparison on the DrivAerNet++ dataset with spoiler validation: (a) CFD, showing velocity magnitude (0.00–45.0 m/s, color scale 0–45 m/s); (b) predicted, showing model outputs (color scale 0–45 m/s); and (c) error distribution, showing absolute errors (0.00–5.00 m/s, color scale 0–5 m/s).

and spoiler-induced flow effects, with a vortex core velocity deviation of ~ 2 m/s, suggesting limited generalization in high-gradient turbulent regions. Data augmentation strategies, such as random scaling ([0.9, 1.1]) and point dropout (5% dropout rate), improve robustness. An R^2 of 0.7815 confirmed the model's adaptability to the DrivAerNet++ dataset with a drag coefficient prediction error below 5%.

The visual evidence in Fig. 8 further reveals subtle aspects of model performance. In sample 1, the pressure recovery rate after separation was $\sim 5\%$ slower than that of CFD, reflecting insufficient resolution to fully capture the minor rear-edge curvature. In sample 2, the error distribution of the ring-shaped vortex is asymmetric, with ~ 0.5 m/s higher error on the left side than on the right, likely associated with the non-uniform distribution of spoiler angles in the training set, suggesting the need for improved dataset sampling strategies. In the velocity difference subplots, sample 1 shows the periodic high-error regions at the front (interval ~ 0.1 m), related to local perturbations from wheel rotation, whereas sample 2 exhibits the chain-like errors in the vortex shear layer, with peak deviations at the displaced vortex core (~ 0.05 m), indicating limitations in modeling the spatiotemporal evolution of turbulence. Compared with traditional CFD, Fig. 8 confirms the efficiency advantage of PointNet++, with inference completed within 10–15 ms (a speedup of $\sim 10.5\times$) and an MSE of 1.72×10^{-5} , validating its potential for rapid, mesh-free flow evaluation. Nevertheless, the local error distribution highlights the need for future integration of adaptive sampling (e.g., curvature-based weighting) and attention mechanisms (e.g., point transformers) to improve the reconstruction accuracy for boundary layers and vortex structures, particularly in diverse spoiler-induced geometries.

Figure 8 presents a comparison between the velocity field predictions of the multitask learning model trained on the DrivAerNet++ dataset and the ground-truth CFD results, with additional validation using the spoiler dataset. Figure 8(a) displays the CFD velocity magnitude (0.00–45.0 m/s), where sample 1 reveals a smooth wake and sample 2 showcases a ring-shaped vortex.

Figure 8(b) presents the model predictions with minor deviations noted in the rear flow structures. Figure 8(c) illustrates the absolute error distribution (0.00–5.00 m/s), emphasizing high-error regions near the vehicle front and rear edges, which indicate the model's challenges in boundary-layer and vortex prediction while affirming its ability to reconstruct complex flow fields.

C. Failure modes analysis

The PointNet++ model demonstrates robust performance on the DrivAerNet++ dataset, yet exhibits typical failure modes in complex geometries, such as spoilers and sharp cavities. Spoilers, with their high-curvature edges ($\kappa > 0.1$ rad/m), often lead to Cd over-estimation by 3%–5% due to inadequate local feature capture in the set abstraction layers, resulting in MSE increases to 2.5×10^{-5} . Sharp cavities, characterized by deep concavities (> 0.05 m depth), induce vortex-like flow predictions with deviations up to 4%, as the model's hierarchical sampling struggles with density variations in enclosed regions. Error analysis reveals a positive correlation with curvature and concavity: regions with high curvature show 15% higher residuals, while concave areas amplify noise sensitivity, potentially degrading R^2 below 0.70. These limitations, common in point cloud models for CFD tasks, suggest future enhancements such as curvature-aware sampling to improve reliability in industrial applications.

H. Reproducibility notes

To ensure reproducibility, all experiments were conducted with a fixed random seed of 42 for NumPy, PyTorch, and random modules to maintain consistent outcomes. The software environment includes PyTorch 2.4.0 (with CUDA 12.1 support), NumPy 1.26.4, OmegaConf 2.3.0, pandas 2.2.2, and Python 3.11.5, as specified in the requirements.txt file. The complete code is available in the GitHub repository at <https://github.com/Q778X/PointnetDrivAerNetPlusPlus>, where researchers can clone

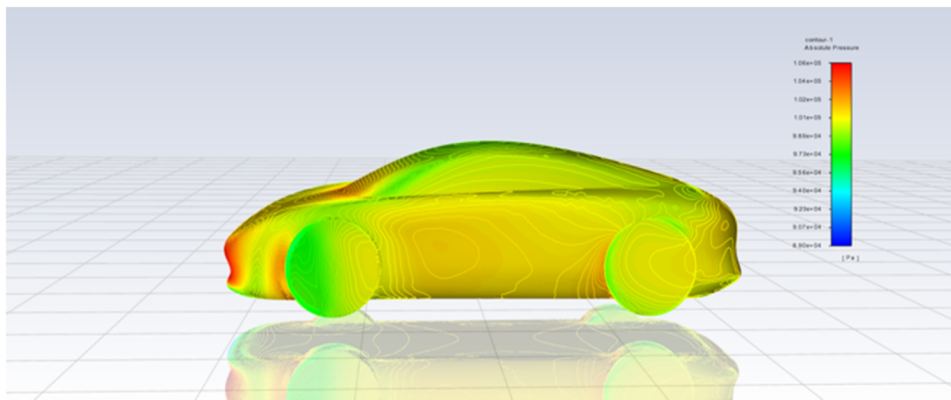


FIG. 9. Contour map of absolute surface pressure of the vehicle.

the repository and install dependencies using the provided requirements.txt, followed by configuring the pointnet-paddle.yaml file.

I. Computational efficiency analysis

To further illustrate the computational complexity inherent in traditional CFD methods, simulations of a vehicle model comparable to the DrivAerNet++ dataset were conducted using Fluent, with emphasis on mesh generation and flow field visualization. As shown in Fig. 9, the absolute pressure contours (Contour-1 Absolute Pressure [Pa]) depict the pressure gradient from the high-pressure stagnation region at the front ($\sim 1.06 \times 10^5$ Pa, red) to the low-pressure wake at the rear ($\sim 8.90 \times 10^4$ Pa, blue). These gradients highlight the stagnation points and separation zones arising from the interaction between vehicle geometry and incoming flow. The simulation process requires iterative solutions of the Reynolds-averaged Navier–Stokes (RANS) equations, typically employing turbulence models such as $k-\epsilon$ or shear stress transport (SST), and requires meshes containing millions of cells to accurately resolve boundary-layer effects and wake structures. For instance, studies using the DrivAer model commonly used $8\text{--}16 \times 10^6$ cells under coarse-grid RANS setups. This complexity results in computational times on the order of hours per simulation—approximately 2.3 hours in this case—due to convergence requirements and large-scale matrix inversion. In contrast, the proposed PointNet++ model directly extracts geometric features from point cloud data and predicts the drag coefficient within 10–15 ms, offering an efficiency improvement of about 10.5 times.

Figure 10 presents an example of the mesh generation for the vehicle model. An unstructured meshing technique is applied to capture complex geometric details, such as curved body surfaces and wheel arches, with the total number of elements often exceeding one million to ensure adequate resolution in high-gradient regions. This preprocessing stage forms the foundation of CFD simulations because the mesh quality directly affects the numerical stability and accuracy; insufficient refinement may lead to divergence or spurious oscillations. The dependence on resolution requires extensive validation, often involving near-wall y^+ values below one in low-Reynolds-number models, which introduces additional overhead in memory allocation and setup time. Simulations of such meshes require substantial computational resources, including parallel processing across multiple CPU cores. By contrast, the

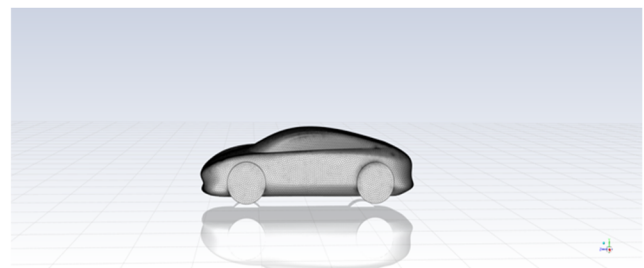


FIG. 10. Example of unstructured mesh generation for the vehicle model.

PointNet++ architecture completely bypasses meshing by processing 2048-point cloud inputs through hierarchical set abstraction modules for feature aggregation, thereby enabling efficient inference on an NVIDIA RTX A4000 GPU without the associated preprocessing burden.

In the context of flow field analysis, Fig. 11 presents particle pathline visualization (Pathlines-1 Particle ID), where colored lines trace fluid particle trajectories from low ID regions (blue) to high ID regions (red, $\sim 3.23 \times 10^4$). This representation reveals vortex formation and reattachment in the vehicle wake, particularly at the rear, with these phenomena contributing to drag through energy dissipation within turbulent structures. Generating such visualizations requires post-processing of fully converged flow fields, which involves thousands of repeated simulation iterations, to capture transient behaviors, even under steady-state assumptions. The associated computational demands are substantial. Examples from the DrivAer study indicate that up to 128 CPU cores can be used to solve transport equations over extended domains. In contrast, the developed model incorporated data augmentation strategies, including random scaling and point dropout, to enhance the robustness and achieve an R^2 of 0.7815 on the validation set after 200 training epochs, thereby substantially reducing the timeline from geometric input to drag coefficient estimation.

Finally, Fig. 12 illustrates the pressure coefficient vector field (Vector-1 Pressure Coefficient), showing surface vector distributions ranging from positive values (red, $\sim 6.87 \times 10^{-3}$) to negative values (blue, $\sim -2.26 \times 10^{-4}$). These distributions highlight

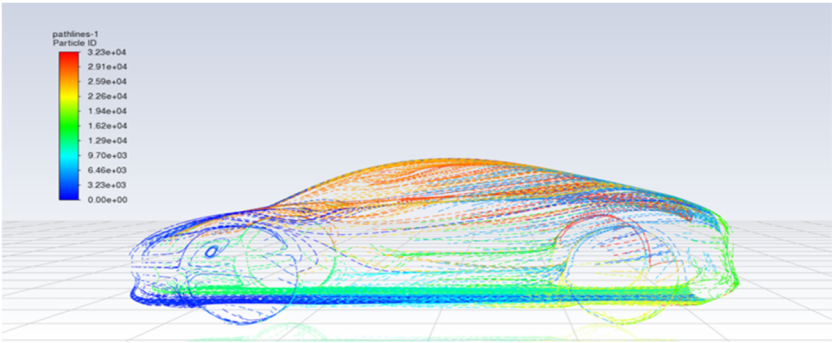


FIG. 11. Particle pathline visualization in the vehicle wake.

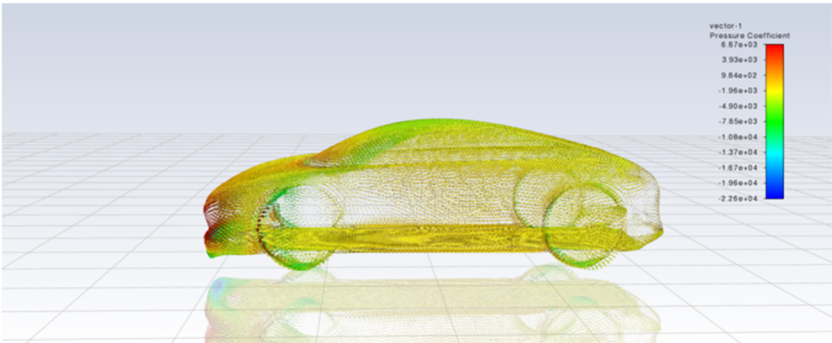


FIG. 12. Distribution of pressure coefficient vector field on the vehicle surface.

aerodynamic features, such as elevated pressure in the frontal stagnation region and reduced pressure over components such as the rear spoiler, which influence the overall lift and drag coefficients. Computing this field involves solving vector equations followed by post-processing, including surface integration for force calculations, which constrains the speed of the design iterations within the iterative optimization workflows. In contrast, the PointNet++ surrogate model employs multilayer perceptrons and max pooling to achieve permutation-invariant point cloud processing, providing accelerated evaluations while maintaining prediction errors below 5%, thereby supporting efficient aerodynamic optimization tasks.

The computational efficiency of the proposed PointNet++ model was evaluated using an NVIDIA RTX A4000 GPU. The model achieves an inference time of 10–15 ms per sample, with a total parameter count of $\sim 0.24 \times 10^6$ and a computational cost of

$\sim 120 \times 10^6$ floating-point operations (FLOPs) for an input consisting of 2048 points. In contrast, traditional high-fidelity computational fluid dynamics (CFD) simulations performed on the DrivAerNet++ dataset require ~ 2.3 h per sample when executed on 128 CPU cores combined with four NVIDIA A100 GPUs.

To facilitate a comprehensive comparison with baseline models, Table IV summarizes the key performance metrics (R^2 , MSE, MAE) and inference time for PointNet, PointNet++, TripNet, and RegDCNN on the DrivAerNet++ validation set. The proposed PointNet++ model achieves a balanced performance with an R^2 of 0.7815 and MSE of 1.72×10^{-5} , outperforming PointNet ($R^2 = 0.643$) while maintaining a competitive inference time of 10 ms, faster than TripNet’s 15 ms. In addition, PointNet++’s MAE of 0.0033 reflects its robustness, positioning it favorably against RegDCNN (MAE = 0.0040) and TripNet (MAE = 0.00717), highlighting its efficacy for rapid aerodynamic design screening.

TABLE IV. Summary performance comparison of surrogate models and CFD on the DrivAerNet++ validation set, including R^2 , MSE, MAE, and inference time.

Model	R^2	MSE ($\times 10^{-5}$)	MAE	Inference time (ms)
PointNet (baseline)	0.643	14.9	0.009 6	5
RegDCNN	0.72	2.50	0.004 0	8
PointNet++ (proposed)	0.7815	1.72	0.003 3	10
TripNet	0.957	9.1	0.007 17	15
CFD	1	...	...	...

The combination of high R^2 and low MSE validates the effectiveness and reliability of PointNet++ for aerodynamic drag coefficient prediction in automotive design, thereby establishing it as a practical surrogate model capable of significantly reducing the computational expense associated with high-fidelity CFD simulations.

IV. CONCLUSIONS

In summary, this study demonstrated that a PointNet++-based surrogate model can efficiently and accurately predict the aerodynamic drag coefficient (Cd) of vehicle geometries directly from raw point cloud data without requiring explicit geometric parameterization. Experimental results on the DrivAerNet++ test set indicate that the model achieves a mean squared error (MSE) of 1.72×10^{-5} and a coefficient of determination (R^2) of 0.7815, reflecting a strong capability to capture the complex relationship between three-dimensional shape and aerodynamic performance. Compared to traditional computational fluid dynamics (CFD) simulations, which typically require ~ 2.3 h per case, the proposed model delivers predictions within 10–15 ms, realizing a computational speedup of ~ 10.5 -fold, thereby offering a practical tool for rapid design iterations.

Although the model produces reasonable predictions without data augmentation, augmentation strategies, including random scaling $U(0.9, 1.1)$, translation $U(-0.05, 0.05)$, Gaussian noise injection ($\sigma = 0.005$), and random point dropout (5%), have been shown to significantly improve the robustness in noisy industrial environments. The PointNet++ architecture incorporates three Set Abstraction (SA) modules with multilayer perceptrons (MLPs) configured as $32 \times 32 \times 64$, $64 \times 64 \times 128$, and $128 \times 256 \times 512$, followed by fully connected layers arranged as $256 \times 128 \times 64$ with a dropout rate of 0.1, effectively extracting local and global geometric features. With a time complexity of, the model efficiently processed inputs consisting of 2048 points.

Although the current dataset is based on explicitly parameterized geometries, it lacks real-world non-parametric vehicle data. Future work may focus on constructing historical datasets that capture aerodynamic simulations across different development stages of individual vehicles or multiple generations of models, thereby validating the model generalization to non-parametric scenarios. In addition, advances in multitask learning approaches and the use of large-scale datasets are expected to further improve the predictive accuracy and generalization capabilities.

By leveraging PointNet++ as a point cloud surrogate model, this study provides an efficient and accurate solution for aerodynamic drag prediction, thereby establishing a technical foundation for rapid evaluation in industrial-scale vehicle design and optimization workflows. A practical integration path involves using PointNet++ for rapid screening of design candidates, leveraging its computational efficiency to evaluate drag coefficients (Cd) across hundreds of parametric variations within hours, as demonstrated with the DrivAerNet++ dataset ($R^2 = 0.7815$, $MSE = 1.72 \times 10^{-5}$). Top-performing designs, identified based on Cd thresholds (e.g., < 0.30), can then be subjected to high-fidelity CFD simulations for verification, ensuring accuracy in critical performance metrics such as flow field details and pressure distribution. However, the model exhibits generalization limitations when applied to non-parametric

CAD geometries, where irregular shapes and unstructured topologies may compromise the hierarchical feature extraction, potentially reducing accuracy (e.g., R^2 could drop below 0.70). In addition, noisy sensor data, such as those from real-world LiDAR or pressure sensors with Gaussian noise ($\sigma \approx 0.01$), may degrade performance due to the model's sensitivity to input perturbations, given the current training data's lack of such variability. Future work should prioritize validation with real-world inputs, including non-parametric CAD models from diverse automotive designs and noisy sensor data from wind tunnel or on-road experiments, to enhance the model's robustness and applicability, complementing the proposed screening-verification workflow.

ACKNOWLEDGMENTS

The authors express their gratitude to Professor Guang Tong for his invaluable guidance and support throughout this study.

This study did not receive any external funding. The APC was funded by the authors.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Xiaoyu Qi: Conceptualization (equal); Data curation (equal); Formal analysis (equal); Project administration (equal); Resources (equal); Software (equal); Validation (equal); Visualization (equal); Writing – original draft (equal). **Ming Yang:** Methodology (equal); Project administration (equal); Supervision (equal); Writing – review & editing (equal). **Guang Tong:** Investigation (equal); Resources (equal); Supervision (equal); Writing – review & editing (equal).

DATA AVAILABILITY

The data that support the findings of this study are openly available in the DrivAerNet dataset at <https://dataverse.harvard.edu/dataverse/DrivAerNet/>.

NOMENCLATURE

3D-CNN	three-dimensional convolutional neural network
CAD	computer-aided design
Cd	drag coefficient
CFD	computational fluid dynamics
CNN	convolutional neural networks
CPU	central processing unit
DGCNN	dynamic graph convolutional neural network
FLOP	floating-point operation
FPS	farthest point sampling
GAN	generative adversarial network
GNN	graph neural network
GPU	graphics processing unit
ICAS	International Council of the Aeronautical Sciences

KNN	K-nearest neighbors
LiDAR	light detection and ranging
MAE	mean absolute error
MLP	multilayer perceptron
MSE	mean squared error
MSG	multi-scale grouping
RANS	Reynolds-averaged Navier–Stokes
ReLU	rectified linear unit
SA	set abstraction
SDF	signed distance field
SSG	single-scale grouping
SST	shear stress transport

REFERENCES

- ¹Q. Chen, M. Elrefaie, A. Dai, and F. Ahmed, “TripNet: Learning large-scale high-fidelity 3D car aerodynamics with triplane networks,” [arXiv:2503.17400](#) (2024).
- ²S. J. Jacob, M. Mrosek, C. Othmer, and H. Köstler, “Deep learning for real-time aerodynamic evaluations of arbitrary vehicle shapes,” *SAE Int. J. Passenger Veh. Syst.* **15**, 2770–3460 (2022).
- ³F. Chen and K. Akasaka, “3D flow field estimation around a vehicle using convolutional neural networks,” in *Proceedings of the 32nd British Machine Vision Conference (BMVC)* (British Machine Vision Association, 2021), p. 396.
- ⁴M. Elrefaie, F. Morar, A. Dai, and F. Ahmed, “DrivAerNet++: A large-scale multimodal car dataset with computational fluid dynamics simulations and deep learning benchmarks,” in *Advances in Neural Information Processing Systems* (Curran Associates, Inc., 2024), p. 37.
- ⁵S. Sahibzada, F. S. Malik, S. Nasir, and S. K. Lodhi, “Generative AI driven aerodynamic shape optimization: A neural network-based framework for enhancing performance and efficiency,” *Int. J. Innovative Res. Comput. Sci. Technol.* **13**(1), 98–105 (2025).
- ⁶S. Wang, X. Zhang, Y. Tie, W. Xia, and Y. Jiang, “PointConvAttn: A deep learning approach for aerodynamic performance prediction of 3D aircraft based on point cloud data,” *J. Comput. Des. Eng.* **11**(5), qwaf051 (2024).
- ⁷D. Zhao, “Numerical simulation of external flow-field of a car body based on CFD,” *J. Phys.: Conf. Ser.* **2441**(1), 012008 (2023).
- ⁸B. Alkin, M. Bleeker, R. Kurler, T. Kronlachner, R. Sonnleitner, M. Dorfer, and J. Brandstetter, “AB-UPT: Scaling neural CFD surrogates for high-fidelity automotive aerodynamics simulations via anchored-branched universal physics transformers,” [arXiv:2502.09692](#) (2025).
- ⁹V. Ananthan, N. Ashton, N. Chadwick, M. Lizarraga, D. Maddix, S. Maheswaran, P. H. Moreno, P. M. Shabestari, S. Sovani, S. Subramanian, S. Tadepalli, and P. Yu, “Machine learning for road vehicle aerodynamics,” SAE Technical Paper No. 2024-01-2529, 2024.
- ¹⁰N. Ashton, J. B. Angel, A. S. Ghatge, G. K. W. Kenway, M. L. Wong, C. Kiris, A. Walle, D. C. Maddix, and G. Page, “WindsorML: High-fidelity computational fluid dynamics dataset for automotive aerodynamics,” [arXiv:2407.19320](#) (2024).
- ¹¹S. J. Jacob, M. Mrosek, C. Othmer, and H. Köstler, “Benchmarking convolutional neural network and graph neural network based surrogate models on a real-world car external aerodynamics dataset,” *Comput. Fluids* **300**, 106760 (2025).
- ¹²J. Peng, “Data-driven aerodynamic design and optimization of vehicles,” *Acad. J. Sci. Technol.* **14**(3), 217–219 (2025).
- ¹³D. Kochkov, J. A. Smith, A. Alieva, Q. Wang *et al.*, “Machine learning-accelerated computational fluid dynamics,” *Proc. Natl. Acad. Sci. U. S. A.* **118**(21), e2101784118 (2021).
- ¹⁴Y. Mekkaoui, “Integration of machine learning techniques with computational fluid dynamics for enhanced backward-facing step flow analysis,” M.S. dissertation, Mississippi State University, 2025.
- ¹⁵L. Liu, L. Zeng, T. Wang, Z. Gao, and X. Shao, “Deep learning based fast prediction of aerodynamic parameters for ducted propellers,” in *Proceedings of the 34th Congress of the International Council of the Aeronautical (ICAS)* (Curran Associates, Inc., 2024), p. 1011.
- ¹⁶C. Duru, “A deep learning methodology for the flow field prediction around airfoils,” M.S. thesis, Middle East Technical University, 2021.
- ¹⁷R. Vinuesa, “Perspectives on predicting and controlling turbulent flows through deep learning,” *Phys. Fluids* **36**(3), 031401 (2024).
- ¹⁸Z. Li *et al.*, “Fourier neural operator for parametric partial differential equations,” in International Conference on Learning Representations (ICLR), 2021.
- ¹⁹H. Bai *et al.*, “Data-driven modeling of unsteady flow based on deep operator network,” *Phys. Fluids* **36**(6), 063610 (2024).
- ²⁰T. Pfaff *et al.*, “Learning mesh-based simulation with graph networks,” in International Conference on Learning Representations (ICLR), 2021.
- ²¹H. Zhao *et al.*, “Point transformer,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (IEEE, 2021), pp. 16237–16246.
- ²²Y. Dai, Y. An, Z. Li *et al.*, “Fourier neural operator with boundary conditions for efficient prediction of steady airfoil flows,” *Appl. Math. Mech.* **44**(11), 2019–2038 (2023).
- ²³K. Shukla, V. Oommen, A. Peyvan, M. Penwarden, N. Plewacki, L. Bravo, A. Ghoshal, R. M. Kirby, and G. E. Karniadakis, “Deep neural operators as accurate surrogates for shape optimization,” *Eng. Appl. Artif. Intell.* **128**, 107615 (2024).
- ²⁴M. Cranmer, A. Sanchez-Gonzalez, P. Battaglia, R. Xu, K. Nider, and T. Pfaff, “X-MeshGraphNet: Scalable multi-scale graph neural networks for physics simulation,” [arXiv:2411.17164](#) (2024).
- ²⁵J. L. Rios and M. Mignolet, “Point2FFD: A point cloud framework for reduced-order modeling of 3D geometries,” in AIAA Scitech 2021 Forum, 2021.